```
  # and a bunch of other stuff
end
```

Later, you can do this:

```
class Book
  def uppercase_title
    title.upcase
  end
end
```

If you declare `class Book` and there is already a `Book` class, Ruby will not error and the new definitions in the second declaration will be added to the existing class. This process of reopening classes to add or change methods is colloquially known as *monkey-patching*.

Typically, you would only extend a class like this if the original class is not part of your code, but it's reasonably common in Ruby to use this method to add utility functions to core classes or the standard library. Ruby on Rails, for example, does this a lot.

To give an example, Ruby on Rails defines a method called `squish`, which clears excessive whitespace in a string, so

```
"This      string     has      whitespace"
```

becomes `"This string has whitespace."` By monkey-patching, Rails can define the method like this :

```
class String
  def squish
    # implementation
  end
end
```

And then call it using `str#squish` just like any other method.

The alternative, which many other languages use, is to define a utility class or classes and define class method on them, which looks like this:

```
class StringUtilities
  def self.squish(str)
    # implementation
  end
end
```

Which you would then call with `StringUtilities.squish(str)`.

Hopefully this example shows the advantage of allowing classes to reopen—the ability to add and easily use utility methods is very convenient. It's nice not to have to know which methods are defined by Rails, and which of the many possible utility classes the method might be in.

That said, this is something to be done with caution—many teams don't allow it in their own code without a very clear reason. And you should be quite wary of using monkey patching to change the behavior of existing methods, rather than adding new methods as we did here. Monkey patching can make the behavior of code unpredictable, it can be hard to tell where behavior is defined, and these changes are global, meaning that if two files